

Spice Garden – AI Restaurant Chat Agent

Project report & technical handover · Generative AI · LangChain · RAG · Tool calling

1. Overview

The Restaurant Chat Agent is a conversational customer-support assistant for a fictional restaurant, **Spice Garden** (Bengaluru). A customer chats in plain language, and the agent decides which internal tool to use to answer accurately. It supports five capabilities:

Capability	How it is delivered
Browse the menu	Lists the full menu or a category; answers dish questions (price, spice, allergens) using RAG
Personalised recommendations	Filters dishes by veg / spice level / budget / allergens; the LLM explains why each pick fits
Order status	Calls a mock order-tracking REST API with the customer's order ID
Table reservations	Checks availability, confirms details with the customer, then books through the mock reservation API
FAQs & policies	Hours, location, parking, payments, cancellation, offers, etc., retrieved from policy documents with RAG

The key design principle is **grounding**: the LLM never answers restaurant facts from its own memory. Every price, timing, policy and order detail comes from a tool, which keeps answers accurate and stops the model from making up facts.

2. Technology Stack

Layer	Technology	Role in the project
Language	Python 3.13 (virtual env)	All components
Large Language Model	Google Gemini <code>gemini-flash-lite-latest</code>	Understands the query, chooses tools, writes the final reply
Agent framework	LangChain 1.x (<code>create_agent</code> , <code>LangGraph</code> runtime)	Tool-calling agent loop, tool schemas, retry middleware
Embeddings	Gemini <code>gemini-embedding-001</code> (3072-dim)	Converts documents and queries into vectors for semantic search
Vector store	FAISS (<code>faiss-cpu</code>), saved to disk	Fast similarity search over 50 knowledge-base chunks
Mock backend API	FastAPI + Uvicorn, JSON files	Simulates the restaurant's order-tracking and reservation services
User interface	Streamlit	Chat window, quick actions, tool-call timeline, API status
Config & testing	<code>python-dotenv</code> , <code>pytest</code>	API key and model settings in <code>.env</code> ; automated API tests

3. Project Structure

<code>gen-ai-project/</code>	
├ <code>data/</code>	<code>menu.json</code> (29 dishes) · <code>policies.md</code> · <code>faq.md</code> ← knowledge base
├ <code>agent/</code>	<code>config.py</code> · <code>rag.py</code> · <code>tools.py</code> · <code>prompts.py</code> · <code>agent.py</code> ← AI logic
├ <code>mock_api/</code>	<code>server.py</code> · <code>orders.json</code> · <code>reservations.json</code> ← mock backend
├ <code>vectorstore/</code>	FAISS index (generated by <code>build_index.py</code>)
├ <code>tests/</code>	<code>test_mock_api.py</code> (pytest) · <code>run_scenarios.py</code> (end-to-end demo)
├ <code>app.py</code>	Streamlit chat UI
└ <code>build_index.py</code>	one-time script that builds the vector store

4. System Architecture

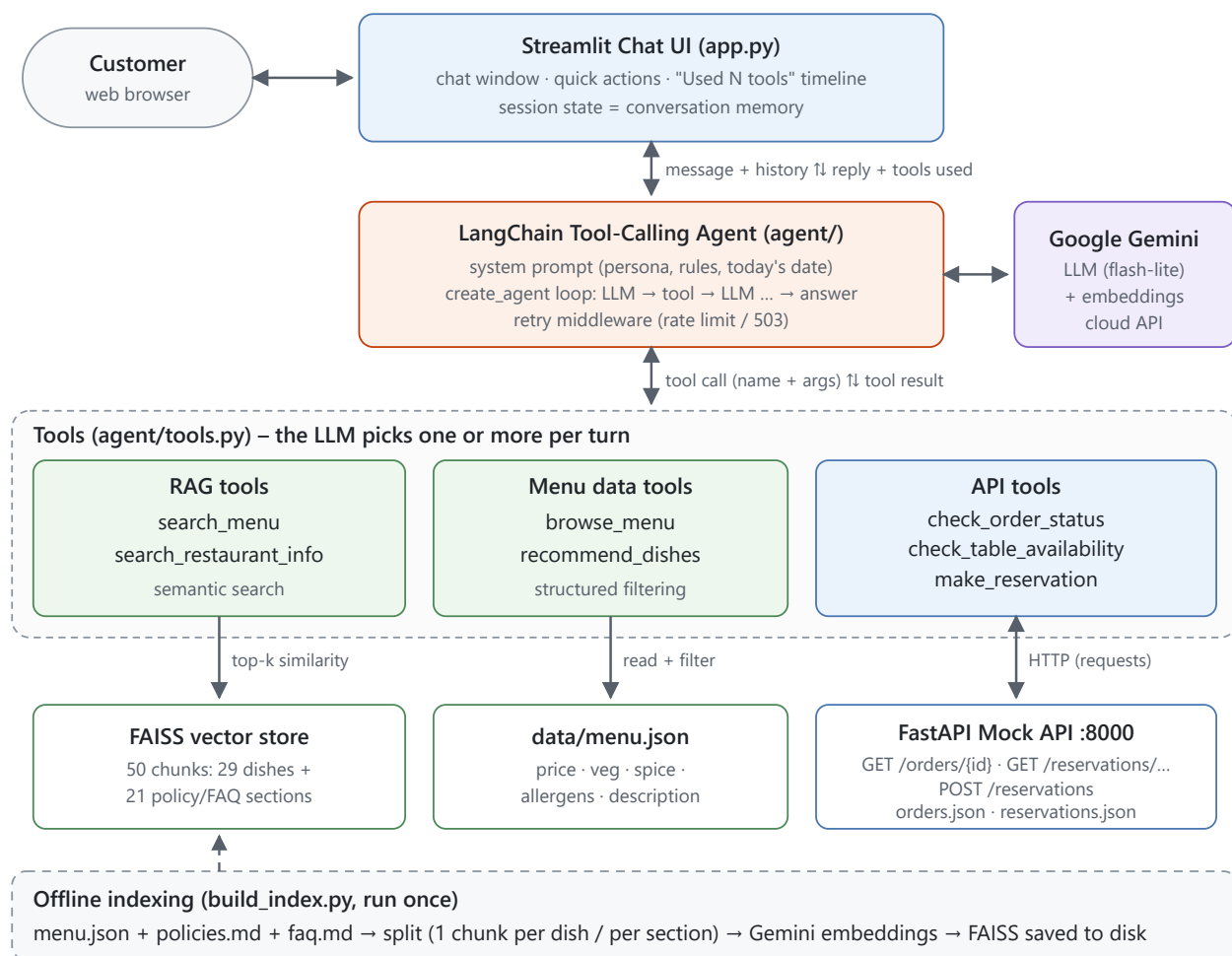


Figure 1 – End-to-end architecture. Solid arrows are the runtime request path; the dashed box is the one-time indexing step.

End-to-end request flow (one chat turn)

- Input** – The customer types a message (or clicks a quick action) in the Streamlit UI.
- Agent call** – `app.py` sends the new message plus the full conversation history to the agent. The history is what gives the agent memory ("Which of *those* are vegan?").
- Reasoning** – Gemini receives the system prompt, the conversation and the schemas of all 7 tools. It either answers directly (greetings, off-topic refusals) or returns a **tool call**, for example `check_order_status(order_id="ORD1003")`.
- Tool execution** – LangChain runs the chosen Python function. It searches FAISS, filters `menu.json`, or calls the FastAPI service over HTTP, and returns the result to the LLM.
- Loop** – The LLM may call more tools (e.g. check availability, then book) until it has enough information.
- Answer** – Gemini writes a short, friendly reply based only on the tool results. The UI shows the reply and a collapsible "Used N tools" timeline listing each tool and its arguments.

5. Components in Detail

5.1 Knowledge base and RAG (`agent/rag.py`)

- Sources:** `menu.json` (29 dishes in 5 categories), `policies.md` (hours, location, reservation, cancellation, delivery, payment, allergy, house rules) and `faq.md` (alcohol, kids, accessibility, buffet, offers, halal, etc.).
- Chunking:** each dish becomes one document (generated from JSON, e.g. "*Paneer Tikka (Starters) – Rs.280. Vegetarian. Spice 2/3. Allergens: dairy...*"). The Markdown files are split by `##` heading, one section per chunk. Each chunk is tagged `kind = menu` or `kind = info`.
- Indexing:** chunks are embedded with Gemini embeddings and stored in a FAISS index on disk. The index is built automatically if missing.
- Retrieval:** the user's query is embedded and the most similar chunks are returned, filtered by `kind` (top 5 for the menu, top 3 for policies/FAQ). These chunks are passed to the LLM as context, which is the *augmented generation* step.

5.2 Agent and prompt (agent/agent.py , agent/prompts.py)

- Built with LangChain's `create_agent(llm, tools, system_prompt, middleware)` using Gemini at temperature 0.3, so replies are consistent and factual.
- The system prompt defines the persona "**Spicy**" and the rules: always use tools for facts, never invent prices or policies, convert relative dates ("tomorrow") using today's date, confirm booking details before booking, and politely decline off-topic requests.
- A **retry middleware** handles Gemini free-tier rate limits (429) and overload errors (503) by waiting and retrying *only the model call*. Tools are never re-executed, so a table is never booked twice.

5.3 Tools (agent/tools.py)

Each tool is a Python function with the `@tool` decorator. Its name, typed arguments and docstring are sent to Gemini as a schema, and that is how the model decides when and how to call it.

Tool	Arguments	What it does
<code>search_menu</code>	<code>query</code>	RAG search over dishes: ingredients, price, spice, allergens
<code>browse_menu</code>	<code>category</code> (optional)	Lists the whole menu or one category from <code>menu.json</code>
<code>recommend_dishes</code>	<code>veg_only</code> , <code>max_spice</code> , <code>max_price</code> , <code>exclude_allergens</code> , <code>category</code>	Filters dishes by preferences; the LLM picks 2–4 and explains why
<code>search_restaurant_info</code>	<code>query</code>	RAG search over policies and FAQs
<code>check_order_status</code>	<code>order_id</code>	GET <code>/orders/{id}</code> → status, items, total, ETA
<code>check_table_availability</code>	<code>date</code> , <code>time</code> , <code>party_size</code>	GET <code>/reservations/availability</code> → free tables or policy error
<code>make_reservation</code>	<code>name</code> , <code>phone</code> , <code>date</code> , <code>time</code> , <code>party_size</code>	POST <code>/reservations</code> → confirmation ID (e.g. RES-6482)

API errors (order not found, slot full, API offline) are returned to the LLM as readable text, so it can explain them politely instead of crashing.

5.4 Mock backend API (mock_api/server.py)

A FastAPI service that stands in for the restaurant's real systems. It has interactive Swagger docs at `/docs`.

Endpoint	Behaviour
GET <code>/orders/{order_id}</code>	Returns one of 8 sample orders (ORD1001–ORD1008 : preparing, out for delivery, delayed, cancelled...) or 404
GET <code>/reservations/availability</code>	Validates the slot against policy and returns tables left (10 per slot)
POST <code>/reservations</code>	Validates, stores the booking in <code>reservations.json</code> , returns a RES-XXXX ID (409 if full)
GET <code>/reservations/{id}</code>	Looks up an existing booking

Reservation rules (same as `policies.md`): 1–10 guests, no past dates, at most 30 days ahead, 30-minute slots within opening hours (weekdays 12:00–15:30 and 19:00–23:00, weekends 12:00–23:30), last slot 1 hour before closing.

5.5 User interface (app.py)

A Streamlit chat app with a welcome message, suggestion chips and sidebar quick actions (menu, recommendation, track order, book table, hours), restaurant info, a live API status badge, and a "Clear chat" button. Each reply shows a collapsible timeline of the tools used, which makes tool calling easy to see during a demo.

6. Example: Table Reservation Flow

```
Customer: "Book a table for 4 tomorrow at 8pm"
→ LLM converts "tomorrow" to 2026-09-27 and calls check_table_availability(2026-09-27, 20:00, 4)
→ API: available, 10 tables left      → Agent asks for name and phone number
Customer: "Rohan Gupta, 9876543210"
→ Agent reads back all details and asks the customer to confirm (no booking yet)
Customer: "Yes, please confirm"
→ make_reservation(...) → POST /reservations → {"reservation_id": "RES-6482", "status": "confirmed"}
→ Agent: "Your table is booked! Reservation ID: RES-6482 ..."
```

If a rule is broken (e.g. 50 guests or 3 AM), the API rejects the request with a reason, and the agent relays it and suggests calling the restaurant.

7. Accuracy and Reliability Measures

- **Grounded answers:** facts come only from tools (RAG or API). If nothing relevant is found, the agent says it doesn't know and shares the phone number.
- **Single source of truth:** menu RAG documents are generated from `menu.json`, so search results and recommendation filters never disagree.
- **Business rules in the backend:** reservation limits are enforced by the API, not left to the LLM.
- **Explicit confirmation:** the availability tool tells the agent to read the details back and wait for a "yes" before calling `make_reservation`.
- **Graceful failures:** rate limits are retried automatically; API or model errors show a friendly message in the UI instead of a crash.

8. Setup, Running and Testing

```
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env                # add GOOGLE_API_KEY (free key: aistudio.google.com)
python build_index.py                  # build the FAISS vector store (once)

uvicorn mock_api.server:app --port 8000 # terminal 1 - mock API (docs: /docs)
streamlit run app.py                   # terminal 2 - chat UI (http://localhost:8501)
```

Test	Coverage
pytest tests	12 unit tests for the mock API: orders, availability, booking, full slots and 6 policy violations
python -m tests.run_scenarios	10 end-to-end conversations with the real LLM: menu, recommendation, allergens, FAQ, order status, unknown order, multi-turn booking, policy violation, memory follow-up, off-topic refusal

Configuration (`.env`): `GOOGLE_API_KEY`, `CHAT_MODEL`, `EMBED_MODEL`, `API_BASE_URL`. After editing anything in `data/`, re-run `build_index.py`.

9. Conclusion

The project shows how Generative AI becomes a practical customer-support system when an LLM is combined with **retrieval** (accurate menu and policy knowledge), **tool calling** (real actions through APIs) and a **conversational interface**. The LLM handles language and reasoning, while facts and business rules stay in reliable data sources and services. This reduces manual workload and gives customers fast, accurate answers.